

מס' שאלון - 532

3

ז' בתמוז תשפ"ה

ביולי 2025

מס' מועד 84

סמסטר 2025ב

20905 / 4

שאלון בחינת גמר
20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה 8 עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
משך זמן הבחינה 3 שעות.
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:

שאלה 3 היא שאלה מפוצלת ל- 2 גרסאות: אחת מיועדת לסטודנטים שלמדו את הקורס בסמסטר 2024, והשניה מיועדת לכל שאר הסטודנטים.
לכן יש להקפיד לענות על השאלה בגרסה הנכונה לפי הסמסטר בו למדתם את הקורס.
סטודנט שיענה על גרסה לא נכונה של שאלה זו, לא יזכה לנקודות.
את תשובותיכם רשמו אך ורק במחברת הבחינה.
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:

ספר הקורס "Essentials of Programming Languages".
מדריך הלמידה "שפות תכנות".

חומר עזר:

ספר הקורס: essentials of programming languages
מדריך הלמידה.
מותר לכתוב הערות בכתב יד בלבד, רק על גבי מדריך הלמידה.
אסור שימוש בעזרים ובחומרים מקוונים.

אינכם חייבים

להחזיר את השאלון לאוניברסיטה הפתוחה

בהצלחה !!!



שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "ויהיי" (שפת LET) המתוארת בספר הלימוד בפרק 3 בעמודים 60-74.

בשאלה זו נוסף לשפת "ויהיי" יכולת של ביצוע המרה (casting) לביטוי. נרצה לתמוך הן בהמרה נרמזת והן בהמרה מפורשת.

נוסיף לשפה ביטוי חדש המבצע casting בצורה מפורשת לערכו של ביטוי נתון. אופן ביצוע ההמרה יוסבר בהמשך.

להלן הדקדוק המגדיר את תחביר הביטוי החדש:

$Expression ::= \langle Type \rangle (Expression)$

cast-exp (typ exp1)

$Type ::= int$

int-type ()

$Type ::= bool$

bool-type ()

- ביטוי cast-exp מבצע המרה של ערכו של exp1 לערך מהטיפוס המתואר ע"י typ, באופן הבא:
 - אם ערכו של exp1 הוא מספרי ונדרשת המרה שלו לערך בוליאני, אזי יוחזר ערך בוליאני של #t אם ערכו המספרי של exp1 שונה מאפס, אחרת יוחזר ערך בוליאני של #f.
 - אם ערכו של exp1 הוא בוליאני ונדרשת המרה שלו לערך מספרי, אזי יוחזר ערך של 1 אם ערכו של exp1 הוא #t, אחרת יוחזר ערך של 0.
 - אם נדרשת המרה של ביטוי לאותו טיפוס של הביטוי – אז לא תבוצע כל המרה ויוחזר ערכו של הביטוי.

בנוסף נרצה לשנות את המימוש של ביטוי if-exp כך שהוא יחושב עם המרה נרמזת של המפרש. כלומר, אם המתכנת ירשום ביטוי מספרי במקום ביטוי בוליאני בחלק התנאי של ביטוי if, במקום שתצא שגיאה, המפרש ידאג לבצע המרה נרמזת של הביטוי המספרי לביטוי בוליאני כפי שהוסבר קודם.

(המשך השאלה בעמוד הבא)

להלן מספר דוגמאות להמחשה:

דוגמה 1:

```
> (run " if 9 then 100 else 200")  
(num-val 100)  
>
```

בדוגמה זו, המתכנת רשם בחלק הבוליאני של ביטוי if ביטוי שאינו בוליאני, והמפרש ביצע המרה נרמזת בעצמו, 9 הומר לביטוי בוליאני של t# ולכן תוצאת התוכנית היא 100

דוגמה 2:

```
> (run " if 0 then 100 else 200")  
(num-val 200)  
> |
```

בדוגמה זו, המתכנת רשם בחלק הבוליאני של ביטוי if ביטוי שאינו בוליאני, והמפרש ביצע המרה נרמזת בעצמו, 0 הומר לביטוי בוליאני של f# ולכן תוצאת התוכנית היא 200

דוגמה 3:

```
> (run "- (100, <int>(zero? (0)))")  
(num-val 99)  
> |
```

בדוגמה זו, בוצעה פעולת חיסור, כאשר הביטוי השני של פעולת החיסור הוא המרה מפורשת של ביטוי בוליאני לביטוי מספרי, במקרה זה t# הומר ל-1, ולכן תוצאת התוכנית היא 99

המשך הדוגמאות בעמוד הבא

דוגמה 4:

```
> (run " if <bool> (9) then 100 else 200")
(num-val 100)
> |
```

בדוגמה זו, בוצעה המרה מפורשת של ביטוי חשבוני לביטוי בוליאני במסגרת ביטוי if.

דוגמה 5:

```
> (run "
    let w = 40
    in
    if <bool> (w) then 500 else 700")
(num-val 500)
> |
```

גם בדוגמה זו בוצעה המרה מפורשת של ביטוי חשבוני לביטוי בוליאני במסגרת ביטוי if.

ממשו והטמיעו את השינויים הדרושים כדי שהוגדרו והודגמו בתוך שפת "ויהי" (שפת LET). בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בחלקי המפרש, ומהם השינויים והתוספות. לא תתקבל תשובה של תיאור מילולי. יש לממש בקוד את השינויים הדרושים.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (}) או פתוח ({)

שאלה 2 (40 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

בשאלה זו נוסיף לשפה 2 ביטויים חדשים:

ביטוי ראשון – ביצוע פעולת map על רשימת ביטויים (בדומה לקיים בשפת scheme). בשאלה זו נרצה שהחישוב של map יבוצע בצורה עצלה (Lazy) – כלומר, פעולת ה-map לא תבוצע בשלב זה אלא רק כאשר יידרש שימוש בתוצאתה היא תחושב לראשונה. נסביר ונדגים זאת בהמשך.

ביטוי שני - ביצוע listref המחזירה את האיבר הנמצא ברשימה באינדקס נתון (בדומה לקיים בשפת scheme)

להלן הדקדוקים המגדירים את תחביר הביטויים החדשים ולאחריהם הסבר על התנהגותם:

$Expression ::= \text{map } Expression [\{ Expression \}^{(*)}]$

map-exp (exp1 exps)

$Expression ::= \text{listref } (Expression , Expression)$

listref-exp (e1 e2)

הסבר הדקדוקים והתנהגותם:

לביטוי map-exp המרכיבים הבאים:

- exp1 - ביטוי שתוצאתו היא פרוצדורה
- exps - רשימה של ביטויים.

ביטוי map אמור לבצע את הפרוצדורה עבור כל אחד מערכי הביטויים ב-exps. את תוצאות ביצוע הפרוצדורה יחזיר map-exp כרשימה.

המפרש ייחשב את הביטוי map-exp בצורה עצלה (Lazy Evaluation) ולא יחשב דבר מהנדרש מביטוי map כעת, אלא בשלב מאוחר יותר בתוכנית כאשר נרצה להשתמש בתוצאת הביטוי. לכן ביטוי map-exp לא יחזיר בשלב זה רשימה כפי שתואר קודם, אלא expval זמני מסוג חדש בשם unevaluated-val אותו יש להוסיף למבנה הנתונים של expval. unevaluated-val ישמור את הפרטים הדרושים (exps, exp1 והסביבה הנוכחית) לצורך חישוב עתידי של פעולת ה-map לכשתידרש תוצאתה.

כאשר תידרש תוצאתה של פעולת ה-map, אז יבוצע חישובה לצורך שימוש בו, ואז יוחזר expval מסוג חדש בשם list-val המתאר את רשימת התוצאות של פעולת ה-map. list-val הוא גם expval חדש שיש להוסיף למבנה הנתונים של expval.

בהמשך יינתנו מספר דוגמאות שימוש עם הסברים.

המשך הסבר הדקדוקים בעמוד הבא

לביטוי listref-exp המרכיבים הבאים:

- e1 - ביטוי שתוצאתו היא רשימה (list-val)
- e2 - ביטוי שתוצאתו היא מספר

ביטוי listref מחזיר את האיבר ברשימה e1 הנמצא באינדקס e2 (האינדקסים הם החל מ-0)

מאחר וביטוי map אינו מייצר את התוצאה הסופית שלו, (מאחר וחישבו עצל), זה עלול להשפיע על ביטויים אחרים בשפה שתוצאתן תלויה בהחזרת ערכו של ביטוי מסוג map-exp, ולכן יש לעדכן בהתאם את ההתנהגות של ביטויים מתאימים כך שריצת התוכנית תחזיר ביטוי מחושב בסופו של דבר. למשל, אם הגדרנו משתנה בשם W השווה לתוצאת ביטוי map-exp, המשתנה W לא יקבל את תוצאת ה-map הסופית (רשימה) אלא תוצאה זמנית המתארת שערכו טרם חושב, רק כאשר תהיה פניה ל-W, יחושב סופית ערכו, ומעתה הוא יהיה רשימה המתארת את תוצאת הפעולה של map. פעם הבאה שתהיה פניה ל-W לא יהיה צורך בחישוב map שוב כי התוצאה כבר חושבה.

להלן מספר דוגמאות להבהרת ההתנהגות הרצויה:

דוגמה 1:

```
> (run "
    let lst= map proc (a) -(a,2) [5, -(10,2) , 80]
    in
      -(listref(lst,2), listref(lst,0))")
(num-val 75)
>
```

בדוגמה זו, הוגדר משתנה בשם lst המקבל תוצאת ביטוי map-exp, בשלב זה lst אינו מייצג את התוצאה הסופית כי ה-map לא מחושב בצורה עצל, כרגע lst הוא תוצאה מסוג unevaluated-val המציגת ערך expval שטרם חושב (ושומרת את כל הפרטים הדרושים לחישוב עתיד). עם משתנה lst כזה מגיעים לחשב את גוף ה-let המורכב מפעולת חיסור. פעולת החיסור תלויה ב-2 ביטויים, הראשון שבהם הוא פעולת listref על הרשימה lst, בניסיון לברר את ערכו של lst מגלים שהוא unevaluated-val ורק כעת מבצעים לו חישוב של הביטוי שהוא מייצג, לקבלת תוצאתו הסופית, שגם מעדכנת את מצבו של lst והוא כעת בעצמו מסוג list-val של רשימת התוצאה שחושבה ((num-val 3) (num-val 6) (num-val 78)), רשימה זו מייצגת את תוצאת פעולת ה-map שבמקרה זה הריץ פרוצדורה להפחתת 2 מכל אחד מאיברי הרשימה שקיבל. בשלב הבא, מחשבים את ערכו של הביטוי השני של פעולת החיסור, שאף הוא מסוג listref, ואף הוא מבצע פניה ל-lst, אך בשלב זה, lst, מייצג תוצאת ביטוי שכבר חושב. פעולת החיסור תבצע חיסור בין האיבר השלישי (אינדקס 2) ברשימה lst (שהוא 78) לבין האיבר הראשון ברשימה (שהוא 3) ולכן הוחזר 75.

המשך הדוגמאות בעמוד הבא


```
> (run "
  let p = proc (a) - (a, 2)
  in
    listref(map p [5, -(10, 2) , 80], 2)")
(num-val 78)
>
```

בדוגמה זו, הוגדרה פרוצדורה p , איתה מגיעים לחישוב גוף ה- let . בגוף ה- let מבוצע $listref$ על רשימה ש- map אמור לייצר, וכפי שהוסבר קודם ביטוי map אינו אחראי על חישובו, כי הוא מחושב בצורה עצלה, לכן לביטוי $listref$ הוחזרה תוצאה מסוג $unevaluated-val$, אך מאחר ונדרשת כעת התוצאה הסופית (הרשימה) $list-ref$ יזם קריאה לחישובו הסופי של ביטוי ה- map מאחר ותוצאתו נדרשת כעת, על מנת שיוכל לחלץ מהרשימה את האיבר המתאים, במקרה זה מאינדקס 2. בדוגמה זו, רואים שילוב של שימוש ב- map עם צורך מידי להשתמש בתוצאתו הסופית.

לכן בכל ביטוי בשפה בו נדרש לחשב ערך של תת-ביטוי אחר, אם התת-ביטוי האחר בעצמו מחושב בצורה עצלה, זה עדיין לא מניב את התוצאה הסופית הדרושה לביטוי ולכן נדרש להפעיל שלב נוסף של חישובה באופן סופי. על כן, יש לבדוק את שאר הביטויים בשפה היכן נדרש לעדכן גם אותם

ממשו והטמיעו את השינויים הדרושים בתוך שפת "וירמוז" (שפת IMPLICIT-REFS).

בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בקבצי המפרש, ומהם השינויים והתוספות.

יש לשים לב ולבדוק מקרים של שימוש לא תקין בתחביר. במקרים של שימוש לא תקין, יש להדפיס הודעות שגיאה מתאימות.

הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל- $value-of$ או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי ($expression$) לבין תוצאת חישוב ביטוי ($expval$) לבין ($identifier$)
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- $terminals, non-terminals$ ופעולות של סגור ($\{ \}$ או $\{ \}^+$)

שאלה 3 (25 נקודות)

שימו לב – זוהי שאלה מפוצלת ל-2 גרסאות שונות. עליכם לפתור את הגרסה המתאימה לסמסטר הלימוד בו למדתם את הקורס. פתרון של גרסה שגויה לא יזכה בניקוד

גרסה עבור סטודנטים שלמדו בסמסטר 2024ב

להלן תוכנית בשפת PROC (שפת "וישגור")

```
let a = 10
in
(if zero?(a) then proc (b) b else proc (c) 50 90)
```

רשמו את שלבי חישוב התוכנית ע"י המפרש תחת סביבה התחלתית ריקה, באמצעות שימוש בסימונים מוסכמים כמתואר למשל בספר הלימוד בעמודים 77-78.
לבסוף רשמו מהי תוצאת התוכנית?

גרסה עבור יתר הנבחנים

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let a = 10
in
(if zero?(a) then proc (b:?) b else proc (c:?) 50 90)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש באלגוריתם שנלמד בפרק 7 בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !